Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №2 по курсу «Информационный поиск»

«Поисковый робот»

Студент: В. О. Горячкин
Преподаватель: А. А. Кухтичев
Группа: М8О-407Б-23
Дата: 19.09.2026
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №2 «Поисковый робот»

Необходимо написать поисковый робот — компоненту обкачки документов.

- Единственным аргументом роботу подаётся путь до yaml-конфига, содержащего данные для базы данных в секции `db`, данные для робота в секции `logic` (задержка между обкачкой страницы) и любые другие данные, необходимые для реализации логики робота.
- Сохранять в базе данных документы с полями: нормализованный url, «сырой» html-текст документа, название источника, дата обкачки в формате Unix timestamp.
- При сохранении оставлять только необходимый текст, удаляя навигационную обвязку.
- Робот можно остановить в любой момент, и при повторном запуске он должен начать с того документа, на котором остановился.
- Периодически робот должен уметь переобкачивать документы, которые уже есть в базе, но только если они изменились.

1 Устройство робота

Работа состоит из двух программ. Обе принимают единственный аргумент — путь до общего yaml-конфига.

- `crawler.py` — обкачка: берёт URL из очереди, скачивает документ, выделяет текст, сохраняет в базу и пополняет очередь найденными ссылками;
- `recrawler.py` — переобкачка: проверяет, не изменились ли уже скачанные документы, и изменившиеся ставит в очередь.

Общий код вынесен в модуль `common.py`: конфиг, нормализация URL, определение кодировки, выделение текста, хранилище и HTTP-клиент.

Хранилище

В качестве базы данных выбран SQLite. Задание называет MongoDB как пример, но SQLite решает ту же задачу и не требует поднимать сервер: модуль `sqlite3` входит в стандартную библиотеку, а вся база — один файл, который удобно приложить к отчёту.

В базе две таблицы. `documents` хранит сами документы:

- `url` — нормализованный адрес, уникальный ключ;
- `raw_html` — «сырой» html-текст документа;
- `text` — выделенный текст без навигационной обвязки;
- `source` — название источника;
- `crawled_at` — дата обкачки, Unix timestamp;
- `content_hash` — отпечаток текста, нужен переобкачке;
- `etag`, `last_modified` — для условных запросов;
- `revision` — сколько раз документ переобкачивался.

Задание требует хранить «сырой» HTML, а лекция — только необходимый текст. Требования не противоречат друг другу, если хранить и то, и другое: `raw_html` пригодится, если правила выделения текста изменятся, а переразбирать корпус по ним, не выкачивая его заново, гораздо дешевле.

Вторая таблица, `frontier`, — это очередь URL: адрес, источник, глубина, статус (`pending`, `done`, `failed`, `skipped`) и приоритет.

Остановка и продолжение

Очередь лежит в базе, а не в памяти. URL помечается как `done` только после того, как документ сохранён, поэтому прерывание в любой момент безопасно: недообработанный адрес остаётся в статусе `pending` и при следующем запуске будет взят снова. Сигналы `SIGINT` и `SIGTERM` перехватываются — робот дописывает текущий документ и выходит, а не умирает посреди записи.

Уникальность `url` в обеих таблицах снимает вопрос дублирования ссылок: повторное добавление просто игнорируется базой.

Нормализация URL

Часть ссылок на странице относительные, поэтому при разборе они склеиваются с адресом текущей страницы. Дальше из адреса убирается всё, что не влияет на содержимое: якорь, служебные параметры (`utm_*`, `gclid` и подобные), порт по умолчанию, регистр схемы и хоста; параметры запроса сортируются, повторные слэши схлопываются.

Отдельно пришлось раскрывать процентные последовательности. Ссылки в HTML Википедии закодированы, и адрес `/wiki/%D0%A4%D0%B8%D0%BB%D1%8C%D0%BC` — это тот же документ, что и `/wiki/Фильм`. Без раскрытия в базе появлялись бы два документа вместо одного.

Определение кодировки

Порядок ровно такой, как на лекции: сначала заголовок `Content-Type`, затем `<meta charset>` в самом HTML, и только потом UTF-8 по умолчанию. Это не теоретическое требование: один из двух источников, `kino-teatr.ru`, отдаёт страницы в windows-1251, и без определения кодировки весь текст с него попал бы в базу нечитаемым.

Вежливость

Робот соблюдает две вещи. Явные правила — `robots.txt`: он скачивается для каждого хоста и кэшируется, запрещённые адреса помечаются как `skipped`. Неявные — пауза между запросами к одному хосту, её величина задаётся в секции `logic` конфига.

Выбор следующего URL

Очередь отдаёт документы так: сначала всё, что ждёт переобкачки, затем — источник, отличный от предыдущего. Чередование источников нужно по двум причинам.

Пока идёт запрос к одному хосту, для другого успевает истечь пауза вежливости, и робот не простаивает. И оба источника наполняются равномерно, а не по очереди целыми сайтами.

Фильтры ссылок

Фильтров два, и это оказалось важным. Первый, `allow`/`deny`, решает, обходить ли ссылку вообще. Второй, `store_allow` и `store_deny`, решает, сохранять ли страницу как документ.

Разделять их пришлось потому, что каталоги, списки и страницы категорий нужны роботу только как источник ссылок. Текста в них хватает, чтобы пройти фильтр по длине, и в первой версии робота база наполнилась оглавлениями вместо статей.

По той же причине в конфиге есть `follow_only` — список страниц, с которых вообще собираются ссылки. Без него робот уходит из своей тематики: из статьи о фильме ссылки ведут на произвольные статьи Википедии, и через несколько шагов корпус перестаёт быть кинематографическим.

2 Переобкачка

Переобкачка вынесена в отдельную программу, `recrawler.py`. Она ничего не скачивает *в базу*: её задача — определить, изменился ли документ, и если да, поставить его URL в очередь. Дальше документ заберёт обычный робот.

Программа берёт из базы документы, которые давно не проверялись (порог задаётся в секции `recrawl` конфига), и по каждому идёт в два шага — от дешёвого к дорогому.

Шаг 1. Условный запрос. Если у документа сохранены `ETag` или `Last-Modified`, запрос отправляется с заголовками `If-None-Match` и `If-Modified-Since`. Ответ `304 Not Modified` означает, что документ точно не менялся, — причём тело страницы сервер при этом не передаёт вовсе.

Шаг 2. Сравнение отпечатка. Если сервер условные запросы не поддерживает, страница скачивается, из неё выделяется текст и считается отпечаток SHA-256. Он сравнивается с сохранённым в базе.

Сравнивается именно *выделенный текст*, а не «сырой» HTML. Это принципиально: в HTML на каждой перезагрузке меняются счётчики, идентификаторы сессий, рекламные блоки и метки времени, и по нему почти любой документ выглядел бы изменившимся. Отпечаток текста меняется только тогда, когда изменилось содержание.

Изменившийся URL попадает в очередь со статусом `pending` и приоритетом 1. Приоритет важен: очередь отдаёт такие адреса раньше новых, поэтому обновление уже известных документов не ждёт, пока разгребётся весь фронт обкачки. При повторной обкачке запись в `documents` обновляется, а счётчик `revision` увеличивается на единицу.

Проверка работы

Цикл проверялся целиком. Сначала переобкачка запускалась на свежескачанных документах — все десять вернули `304 Not Modified`, в очередь не попал никто. Затем у двух документов в базе отпечаток был подменён вручную, что имитирует изменение страницы на сайте. Повторный запуск нашёл ровно эти два документа и поставил их в очередь:

```
[изменился] https://ru.wikipedia.org/wiki/?_(фильм)
[изменился] https://ru.wikipedia.org/wiki/...а_пятый_всадник_-_Страх

Проверено документов:      6
ответ 304 Not Modified:  4
текст совпал:            0
изменились:              2
```

```
В очередь на переобкачку добавлено: 2
```

Запущенный следом робот взял оба адреса первыми, до новых документов, и поднял им `revision` с 1 до 2:

```
[updated ]   15790 симв.  https://ru.wikipedia.org/wiki/?_(фильм)
[updated ]    3011 симв.  https://ru.wikipedia.org/wiki/...а_пятый_всадник_-_Страх
[new     ]    1863 симв.  https://ru.wikipedia.org/wiki/0-41*
```

3 Результаты

Робот запускался на корпусе «кино и сериалы» с двух источников. Прогон ограничивался четырьмястами сохранёнными документами: цель прогона — измерить скорость и проверить работу всех механизмов, а не набрать полный корпус. Итоги сведены в таблицу 1, разбивка по источникам — в таблицу 2.

Что собрано

Показатель	Значение
Сохранено документов	400
Обойдено URL	933
Осталось в очереди	3 221
Объём «сырых» данных	81,2 МБ
Объём выделенного текста	3,02 МБ
Доля полезного текста	3,7 %
Средний размер документа	207,9 КБ
Средний объём текста	7 918 символов
Размер файла базы	87,6 МБ

Таблица 1: Итоги прогона робота

Распределение объёма текста сильно скошено: медиана 4 376 символов при среднем 7 918, девяностый перцентиль 18 934, максимум 81 380. Это согласуется с оценкой распределения, полученной в первой лабораторной работе.

Источник	Док.	Сырой, КБ	Текст, симв.	Кодировка
ru.wikipedia.org	383	212,2	7 326	UTF-8
www.kino-teatr.ru	17	109,8	21 240	windows-1251

Таблица 2: Документы по источникам

Определение кодировки отработало: 383 документа сохранены как UTF-8, 17 — распознаны как windows-1251 и приведены к единому виду.

Скорость

При паузе в одну секунду робот выдаёт около 0,95 страницы в секунду. Чередование источников почти удваивает пропускную способность: пока идёт запрос к одному хосту, пауза для второго истекает, поэтому суммарная скорость близка к одному запросу в секунду, а не к одному запросу в две секунды.

Из этого следует оценка для полного корпуса. На 30 тысяч документов при таком темпе уйдёт порядка 9–10 часов, и это без учёта страниц, которые скачиваются ради ссылок, но документами не становятся, — в нашем прогоне таких оказалось больше половины (933 обойдённых URL против 400 сохранённых). Реальная оценка — около суток непрерывной работы. Робота можно останавливать и продолжать, так что это не препятствие, но однопоточность — главный ограничитель.

Блокировка второго источника

Главный результат прогона отрицательный, и его стоит описать подробно.

Из 119 обойдённых адресов документов Кино-Театр.Ру сохранились лишь 17. Остальные 102 вернули не документ, а страницу одинакового размера в 1041 символ с текстом:

```
Кино-Театр.Ру Роботам запрещено посещать наш сайт.
Пожалуйста,подтвердите,что вы не робот. Проверка: введите...
```

Сайт закрыт сервисом ddos-guard, и после примерно 470 запросов с паузой в одну секунду защита перевела робота на капчу. Риск был отмечен ещё в первой лабораторной работе, но тогда проверка пятью запросами прошла успешно и масштаба проблемы не показала.

Неприятнее всего то, что робот не заметил подмены. Формально ответ приходил с кодом 200, и страница с капчой — такой же документ, как любой другой. От попадания в базу четырёхсот одинаковых заглушек спас случай: порог `min_text_length` в 1500 символов отсёк их как слишком короткие. Если бы порог был ниже, корпус оказался бы испорчен незаметно.

По итогам в робота добавлены две вещи:

- `blocked_marker` — признак страницы-заглушки в конфиге источника. Страница с этим признаком не сохраняется, а URL помечается в очереди как `failed` с указанием причины;
- своя пауза для источника: у Кино-Театр.Ру она увеличена до 5 секунд, что соответствует директиве `Crawl-delay` в его `robots.txt`.

Проверка показала, что распознавание работает: повторный запуск честно сообщает `[блокировка]` вместо того, чтобы молча портить базу. Но сама блокировка на момент написания отчёта не снята — сайт отдаёт капчу даже на страницу каталога. Вопрос остаётся открытым: нужно либо выждать и качать существенно медленнее, либо заменить второй источник на более терпимый к роботам.

4 Выводы

Робот написан и работает: обходит два источника, сохраняет документы в базу, переживает остановку и умеет обновлять то, что изменилось. Переобкачка вынесена в отдельную программу и связана с роботом через очередь.

Самое полезное, что дала работа, — это несколько ошибок, которые на бумаге не видны.

Фильтр «что обходить» и фильтр «что сохранять» — это разные фильтры. В первой версии робот складывал в базу оглавления и каталоги: текста в них достаточно, чтобы пройти проверку по длине, и формально они проходили все условия. Понадобился отдельный набор правил, решающий, что считать документом, а что — только источником ссылок.

Без ограничения на то, откуда брать ссылки, корпус перестаёт быть тематическим. Из статьи о фильме ссылки ведут на статьи о городах, годах и профессиях, и через несколько шагов обхода робот собирает уже не кино. Пришлось разрешить сбор ссылок только со страниц категорий и каталогов.

Стандартная библиотека умеет подвести молча. Робот отказывался качать Википедию целиком, помечая все адреса как запрещённые. Причина оказалась в том, что `RobotFileParser.read()` ходит за `robots.txt` через `urllib` со стандартным User-Agent, Викимедиа отвечает на него `403`, а парсер трактует `403` как «запрещено всё». Исправление — качать `robots.txt` той же сессией, что и остальные страницы, и разбирать его через `parse()`.

Адрес нужно нормализовать раньше, чем кажется. Ссылки в HTML Википедии процентно закодированы, и пока адреса не раскрывались, правила с кириллицей по ним не срабатывали: в базу лезли служебные страницы, которые фильтр должен был отсечь.

Недостатки

Робот однопоточный. Пауза вежливости в секунду означает предел около двух документов в секунду на два источника, и на корпус в 30 тысяч документов уйдут часы. Чередование источников частично это скрадывает, но настоящее решение — несколько потоков с очередью на каждый хост, как в схеме URL frontier с лекции.

«Сырой» HTML хранится без сжатия, и на него уходит на порядок больше места, чем на текст. Текст сильно избыточен и сжимается в разы, так что сжатие здесь напрашивается.

Переобкачка обходит документы по времени последней проверки, то есть одинаково часто для всех. Разумнее было бы учитывать, как часто документ менялся раньше, и проверять стабильные страницы реже, а живые — чаще.

Наконец, робот не защищён от ловушек: календари и страницы с бесконечной пагинацией порождают бесконечную очередь. Пока от этого спасают только ограничение глубины и фильтры адресов.

Список литературы

[1] Маннинг К. Д., Рагхаван П., Шютце Х. *Введение в информационный поиск* — М.: Издательский дом «Вильямс», 2011. — 528 с. — ISBN 978-5-8459-1623-4.

[2] The Robots Exclusion Protocol [Электронный ресурс]. — URL: https://www.robotstxt.org/robotstxt.html (дата обращения: 19.09.2026).

[3] RFC 7232. Hypertext Transfer Protocol (HTTP/1.1): Conditional Requests [Электронный ресурс]. — URL: https://www.rfc-editor.org/rfc/rfc7232 (дата обращения: 19.09.2026).